State, Not Syntax: Harness Optimization for Small-Model Multi-Turn Tool Calling

Sanskar Thapa

Master's thesis draft (presentation end of November 2026)

`https://github.com/sskarz/Thesis`

October 2026

Abstract

Assistants that *act* do so by emitting *tool calls* that a surrounding program, the *harness*, executes. Running that loop locally with a small language model keeps data private and costs nothing per request, but small models fail when a task needs several dependent calls. We study why on the multi-turn base split of the Berkeley Function Calling Leaderboard (BFCL v4; 200 tasks graded by the final state of a simulated backend), using Gemma-4-E2B-it served on a MacBook Pro. The failure is *state*, not *syntax*. The model scores 83.1% on single-call selection yet completes only 27.0% of multi-turn tasks through the native tool API, and forcing syntactically valid output makes it worse (10.5% and 13.5%). Optimizing the harness helps with no weight updates: JSON-in-text calls reach 30.5%, a three-part minimal recipe 35.5%, and the full stack—JSON text, a harness-written state summary, a gated stop-on-repeat rule and a four-line status card—77/200 (38.5%; +8.0 points, McNemar $p = 0.023$ uncorrected; not significant after Holm correction), on par with a five-sample majority vote (36.5%) at about a seventh of its generation time. Training did not reach that level: the best supervised fine-tune scored 15.0% ($p < 0.001$ below JSON text) and preference tuning 26.5%. A null test clears the training pipeline; the data mix remains the open suspect.

1 Introduction

A useful assistant has to act, not only chat. To check a calendar or lock a car, a language model emits a *tool call*: a machine-readable request (a function name plus arguments) that a surrounding program—the *harness*—runs against a *backend*, here a simulated vehicle or filesystem, and whose result is fed back to the model (Figure 1). Real tasks need several dependent calls in a row (*multi-turn* tool calling): look up a value, use it, check the outcome, and know when to stop.

Running this loop locally with a *small language model* (SLM) keeps private data on the device, works offline, and has no per-request cost. The catch is accuracy. We measure Gemma-4-E2B-it [1], a two-billion-effective-parameter model, on BFCL v4 [2], served with llama.cpp [3] on a MacBook Pro. Out of the box, through the native tool interface, it completes 27.0% of 200 multi-turn tasks.

We treat this as a *harness optimization* problem. The model weights stay fixed; what we change is the loop around the model: how calls are parsed, what state the harness records and shows back, when the harness stops a turn, and how the next step is framed. Two of these levers (the state summary and the status card) still reach the model as text in its context, so this is not "prompt-free": it is control computed by the harness from executed results, delivered partly through the context. We contrast it with two other families of fixes: forcing the model's output to be syntactically valid, and changing the weights by fine-tuning. Our contributions:

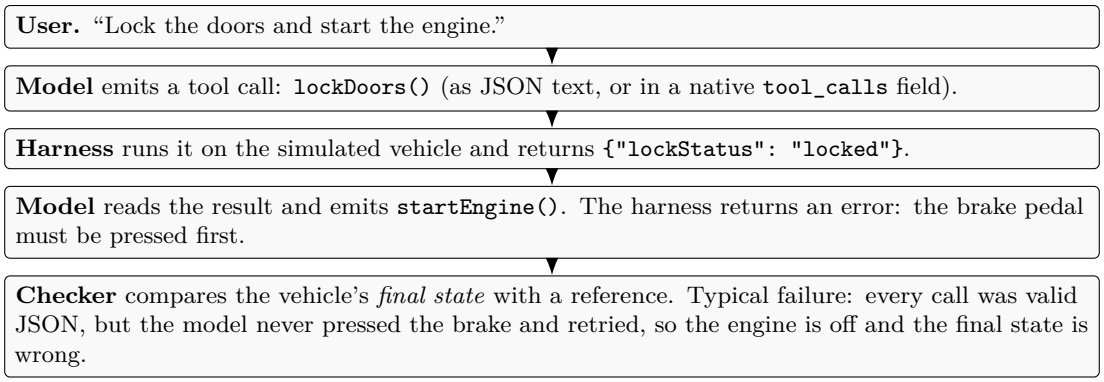


Figure 1: The multi-turn tool-calling loop studied in this paper. Grading is by the final state of the simulated backend, not by whether the calls parse. Illustrative, not a scored task.

1. **A within-model failure anatomy: state, not syntax.** The same model, quantization and server score 83.1% on single-call selection and 27.0% on multi-turn tasks. Once calls parse, failures are tools that never ran and worlds that end in the wrong state, not malformed output; decoders that *guarantee* valid output drop accuracy to 10.5% and 13.5% (Section 4).

2. **A training-free harness ladder, with paired statistics, that is also the cheap option.** JSON-in-text 30.5% → minimal recipe 35.5% → full stack 38.5% (77/200), +8.0 pp over JSON text, greedy throughout, and on par with five-sample majority voting (36.5%) at about 1/7 of the generation time. The status card works as an act-versus-abstain control rather than a state-tracking aid (Section 5).

3. **A controlled negative result on fine-tuning a 2B tool caller.** No trained model beat the untuned model: best supervised fine-tune 15.0% ($p < 0.001$ below JSON text, surviving correction), preference tuning 26.5% (within noise). Models trained on a rebuilt mix collapse on a 20-task gate, while a null test stays near the bar, which clears the training pipeline and points to the data (Section 6).

2 Background and Related Work

Benchmark. The Berkeley Function Calling Leaderboard (BFCL) evaluates language models as API callers [2, 4]. Its single-turn categories check one emitted call against a reference with an abstract-syntax-tree (AST) matcher. A *turn* is one user request and everything the model does before handing control back. Its multi-turn categories run the model in an agent loop against simulated backends (for example VehicleControl and GorillaFileSystem, a sandboxed Unix-like filesystem) and compare the backends' final state to a reference. We use the *multi-turn base* split (200 tasks, all tools available). Accuracy is the fraction of the 200 tasks whose final state matches; it is not the public overall BFCL score.

Output channel. With *native function calling* (NFC), the server receives a tool list and the model returns structured `tool_calls`, as in most production APIs. With *instruction-followed function calling* (IFFC), the model is asked in the prompt to write its calls as a JSON list in ordinary text, which the harness parses. Taheri et al. report that small models follow such JSON instructions more reliably than native tool APIs on single-turn BFCL [5]; we use their single-model variant.

Constrained decoding. Grammar-constrained generation restricts the next token so that output is always valid, for example to a JSON schema or a list of tool names [6, 7]; Yu et al. constrain tool names with a prefix-tree automaton [8]. Format restrictions can also reduce answer quality [9].

Harness-side control and test-time compute. Context-compression methods summarize long histories [10]; our state summary is instead written by the harness from executed results. Typed observe–orient–decide–act (OODA) controllers structure agent steps [11]; our status card borrows only the name. Guided templates add a reasoning scaffold before calls [12]. Sampling several answers and voting is a standard way to buy accuracy with compute [13].

Training for tool use. Specialist small callers such as xLAM [14] and Hammer [15] are trained on synthetic tool data such as ToolACE [16] and APIGen [17]. *Supervised fine-tuning* (SFT) trains the model to reproduce target responses; we use low-rank adapters (LoRA) [18] with MLX [19]. *Direct Preference Optimization* (DPO) trains on pairs of preferred and rejected responses [20, 21].

3 Setup and Methods

Model and serving. All results use Gemma-4-E2B-it, quantized with Q4_K_M (a 4-bit weight format of llama.cpp), served by `llama-server` (llama.cpp build 8111) with Metal acceleration on a MacBook Pro (Apple M4 Pro, 48 GB). Decoding is greedy (temperature 0) except for majority voting. Most runs used four parallel server slots and a 65,536-token context; the native-API baseline used eight slots, a difference we return to in Section 7.

Harness interventions. All leave the weights untouched and use the same checker (Appendix A gives exact rules and examples):

- **JSON in text (IFFC)** instead of the native tool API (NFC); the harness parses the calls.
- **State summary (scratchpad).** After each executed call, the harness writes a short `[STATE]` snapshot of facts from real results (current directory, known files, last calls) into the conversation.
- **Stop on repeats.** The harness ends a turn when the model re-proposes calls it has already executed, but only after at least six tool steps in the current request. An *ungated* variant stops at the first repeat.
- **Status card.** A four-line card (goal, known facts, last call and outcome, and an advisory next move: call, ask, finish or refuse) is prefixed to the latest user message. The harness fills it deterministically from the conversation and executed results; it never sees reference answers. JSON text + stop on repeats + card is our *minimal recipe*.
- **Full stack (headline).** JSON text + state summary + stop on repeats + status card. Greedy; no sampling.
- **Vote of five (Best-of-N, BoN).** At each step, sample five candidate calls at temperature 0.7, execute the most common one; on top of the state summary. This is the compute-heavy comparison point.
- **Forced-valid decoding.** *Constrained decoding* restricts output in two passes (tool names, then arguments by schema); a *tool-name automaton* constrains only the name, inspired by [8].

Training. We train LoRA adapters on Gemma-4-E2B-it with MLX, merge, quantize to Q4_K_M and evaluate with JSON text. Early mixes combine ToolACE dialogues with sampled xLAM data; a later *rebuilt mix* (30,000 rows) combines about 45% newly simulated, execution-checked episodes with cleaned public data. We also train one DPO model. Because a 200-task run takes about three hours, trained models first face a fixed 20-task *gate* drawn from the same split (bar: 17/20, the untuned model with the card); a smaller 10-task regression check is described in Appendix B.1.

Error labels. Each failed task gets one primary label from the checker's message: *empty turn* (no call on a turn that needed one), *under-call* (a required tool never ran), or *state mismatch* (tools ran but the backend's final state differs from the reference).

Statistics. We report Wilson 95% intervals, which stay accurate for small counts. For paired comparisons on the same 200 tasks we use exact two-sided McNemar tests, which count only the tasks where two conditions disagree, from per-task results in our evaluation database. We ran eight planned tests; p-values are uncorrected, and we note where a Holm correction for eight tests changes the conclusion.

4 Where Small Models Fail

One call is easy; a chain is not. On single-call BFCL categories (simple, multiple, irrelevance, Java and JavaScript; 990 cases), Gemma-4-E2B-it through the native API scores 823/990 (83.1%, interval 80.7–85.3%; Appendix C). On multi-turn tasks, the *same model, quantization and server* completes only 54 of 200 (27.0%). The gap is between choosing one call and carrying a chain of calls through a changing state.

After calls parse, errors are about the world. Figure 2 breaks down failed tasks. With JSON text, the two biggest buckets are state mismatches (60) and under-calls (38): tools that should have run never did, and the ones that ran left the world wrong. The native API fails differently: its main non-state error is empty turns (59), and under-calls are its smallest bucket (19). For every JSON-text condition in Figure 2, state mismatch is the largest bucket (53–109 tasks); with the state summary, 22 of its 64 state mismatches are in VehicleControl and 21 in the filesystem. The full stack has 20 empty turns, 35 under-calls and 68 state mismatches.

Forcing valid syntax hurts. If failures were mostly malformed output, guaranteeing valid output should help. It does the opposite. Two-pass constrained decoding scores 21/200 (10.5%; interval 7.0–15.5%), 20.0 pp below JSON text, with 70 under-calls and 95 state mismatches. The tool-name automaton scores 27/200 (13.5%; 9.4–18.9%), with state mismatch rising to 102. Our two-pass decoder approximates, rather than reproduces, full grammar engines [7], but the effect is large and in the wrong direction: a valid call is not a correct next step.

5 What Helps: Harness-Side Control

Figure 3 and Table 1 show the main results. Every change is greedy except the vote of five, and none touches the weights.

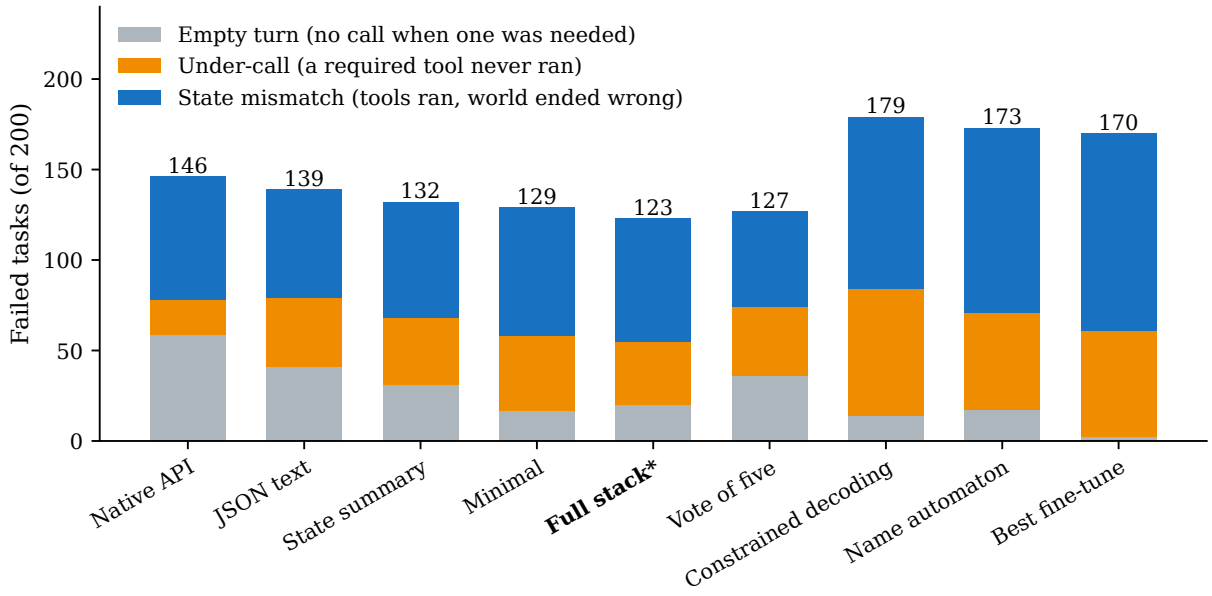


Figure 2: Why tasks fail. Each bar is one condition on the same 200 tasks (* marks the headline full stack); height is the number of failed tasks, split by primary error. Harness control mostly removes empty turns (grey); wrong final states (blue) remain. Forced-valid decoders and the best fine-tune shift failures into under-calls (orange) and state mismatches.

Table 1: Main multi-turn results (200 tasks). Interval: Wilson 95%. Δ: difference from JSON text in percentage points (pp). Full scoreboard in Appendix B.

Condition	Correct	Acc.	95% interval	Δ (pp)
Output format				
Native tool API (NFC)	54	27.0%	21.3–33.5	−3.5
JSON in text (IFFC)	61	30.5%	24.5–37.2	—
Harness control on JSON text				
+ stop on repeats	65	32.5%	26.4–39.3	+2.0
+ state summary	68	34.0%	27.8–40.8	+3.5
+ stop on repeats + status card (minimal recipe)	71	35.5%	29.2–42.3	+5.0
+ summary + stop + card (**headline; full stack**)	77	**38.5%**	32.0–45.4	+8.0
+ summary + vote of five (compute-heavy, ∼7× time)	73	36.5%	30.1–43.4	+6.0
Forced-valid decoding				
Constrained decoding (two-pass)	21	10.5%	7.0–15.5	−20.0
Tool-name automaton	27	13.5%	9.4–18.9	−17.0
Training				
Best supervised fine-tune	30	15.0%	10.7–20.6	−15.5
Preference-tuned (DPO)	53	26.5%	20.9–33.0	−4.0

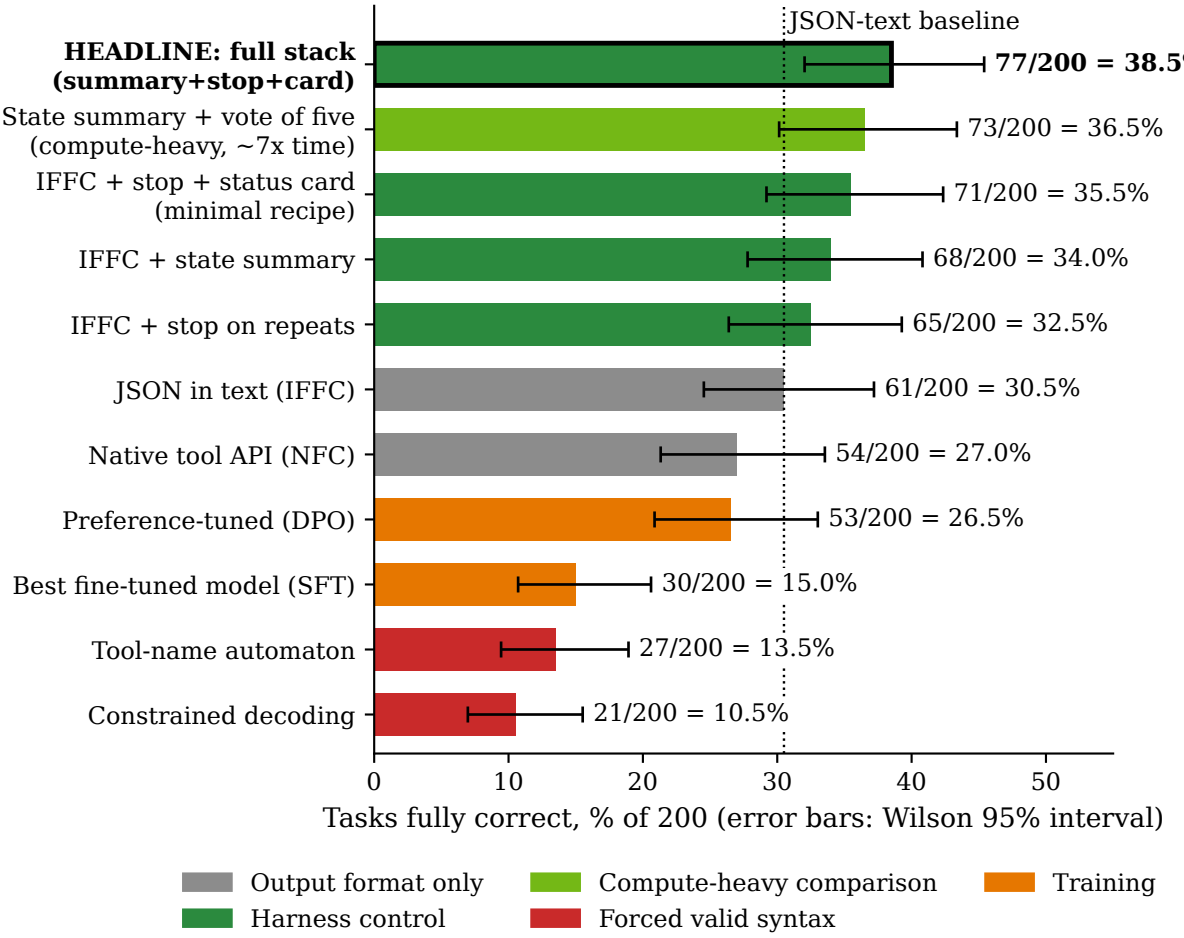


Figure 3: Multi-turn accuracy on 200 BFCL v4 tasks, Gemma-4-E2B-it. Bars: share of tasks fully correct; error bars: Wilson 95% intervals. Green: harness control (headline full stack outlined, in bold); light green: majority vote, the costly comparison; grey: output format alone; red: forced-valid decoders; orange: training. Dotted line: JSON-in-text baseline.

JSON text over the native API (+3.5 pp). Writing calls as JSON text scores 61/200 versus 54/200 natively; paired, it solves 22 tasks the native run misses and loses 15 ($p = 0.32$, not significant). Part of the gap may come from serving (eight slots versus four), and a guided template on the native API ties it (Section 7). We use JSON text as the base for the harness ladder because it is what the harness can parse and inspect most easily, not because the channel switch alone is established.

State summary (+3.5 pp). The harness-written snapshot scores 68/200; versus JSON text it gains 20 tasks and loses 13 ($p = 0.30$). Empty turns fall (41 to 31) while state mismatches do not (60 to 64), so the summary mainly keeps the model calling.

Stop on repeats and the status card (minimal recipe, 35.5%). Stopping on a repeated call after six tool steps scores 65/200 (+11/−7 versus JSON text, $p = 0.48$). Stopping at the *first* repeat instead scores 54/200: all 37 forced stops were on tasks that failed, so the gate matters. Adding the

status card scores 71/200, +5.0 pp over JSON text (+26/−16, $p = 0.16$). The card is best read as an act-versus-abstain control: empty turns fall from 38 to 17, while state mismatches rise from 60 to 71. It helps the model act, not track state.

Full stack: the best measured result (38.5%). Combining all four greedy changes scores 77/200 (38.5%, interval 32.0–45.4%) in about 2.9 hours for 200 tasks. Paired against JSON text it gains 30 tasks and loses 14 ($p = 0.023$ uncorrected; not significant after Holm). Against the minimal recipe, the state summary alone and the vote of five the differences are within noise (Table 2). This configuration was assembled *after* we had seen earlier results on the same 200 tasks, so its lead is subject to tuning leakage. Its harness code is on draft pull request #90 (commit `c894ec6`), run with the same model, quantization and server.

Cost: majority vote (36.5%). Voting over five samples on top of the state summary scores 73/200, four tasks fewer than the greedy full stack (+24/−20 for the full stack, $p = 0.65$). It costs about 7× the generation wall-clock (21.3 hours versus 3.0 for the state summary alone and 2.9 for the full stack, 200 tasks), from a single sampling seed. A greedy harness matches it at a fraction of the time, which matters on a local device.

Table 2: Exact two-sided McNemar tests on per-task results (200 tasks each, same task IDs). "A only" and "B only" count tasks that one condition solves and the other misses. p is uncorrected; "Holm" marks whether it stays below 0.05 after a Holm correction for the eight planned tests (rows 1–8).

A	B	A only	B only	p	Holm
Full stack (38.5%)	Minimal recipe (35.5%)	19	13	0.38	no
Full stack	Vote of five (36.5%)	24	20	0.65	no
Full stack	State summary (34.0%)	24	15	0.20	no
Full stack	JSON text (30.5%)	30	14	0.023	no
Minimal recipe	JSON text	26	16	0.16	no
Minimal recipe	Native API (27.0%)	37	20	0.033	no
JSON text	Native API	22	15	0.32	no
Best SFT (15.0%)	JSON text	7	38	< 0.001	yes
State summary	JSON text	20	13	0.30	—
Vote of five	State summary	17	12	0.46	—
Stop on repeats (32.5%)	JSON text	11	7	0.48	—
Minimal recipe	Stop on repeats	26	20	0.46	—

How sure are we? Every interval in Table 1 is about ±6–7 pp wide. Among harness conditions, only the full stack versus JSON text ($p = 0.023$) and the minimal recipe versus the native API ($p = 0.033$) fall below 0.05, and neither survives Holm correction. The ordering among the harness rows is therefore observed, not proven; the +8.0 pp is suggestive, not established. Two contrasts are robust. The best supervised fine-tune solves only 7 tasks that JSON text misses while missing 38 that it solves ($p < 0.001$, surviving correction), and the full stack's interval (32.0–45.4%) does not overlap those of the best fine-tune (10.7–20.6%) or the forced-valid decoders. Preference tuning (26.5%) is *not* separable from JSON text.

6 Why Fine-Tuning Did Not Help

Fine-tuning is the obvious way to teach a small model a task. Here no trained model beat the untuned model. We treat the runs as evidence about *what* goes wrong; the full version history is in Appendix B.1.

Public data taught the model to stop; repairs restored calling, not accuracy. The first mixes scored 1/200 (0.5%). A *stop target* is a training response that is an empty list ("no further call"). After a tool result, 78.1% of targets in that data were stop targets (2,121 versus 596 continue), all from ToolACE, so the model learned to stop after its first tool result while BFCL needs further calls. Removing those targets and adding recovery examples scored 30/200 (15.0%), our best fine-tune: empty turns nearly vanished (167 to 2), but failures moved to state mismatch (109) and under-call (59). Adding the state summary to this fine-tune *lowered* it to 26/200, although the summary helps the untuned model. A DPO model scored 53/200 (26.5%), within noise of JSON text.

A rebuilt mix collapsed; a null test clears the pipeline. The best-validation checkpoint of the rebuilt mix scored 2/20 on the gate (Figure 4); validation loss did not predict the collapse. It usually made one call and stopped, even when a setup step was needed (looking up a stock symbol, changing directory, logging in), and invented missing values such as an order number. Removing the card scored 1/20, and continued training on test-style tool layouts still scored 2/20. Was the pipeline the problem? A *null test* should change nothing if the machinery is sound: we fine-tuned the untuned model on 156 of its own logged answers through the same template, adapter, merge, quantization and serving steps. It scored 15/20 (interval 53–89%), within noise of the bar (17/20; 64–95%), while the rebuilt mix's 2/20 (3–30%) is far outside it. The collapse came from *what* the mix taught.

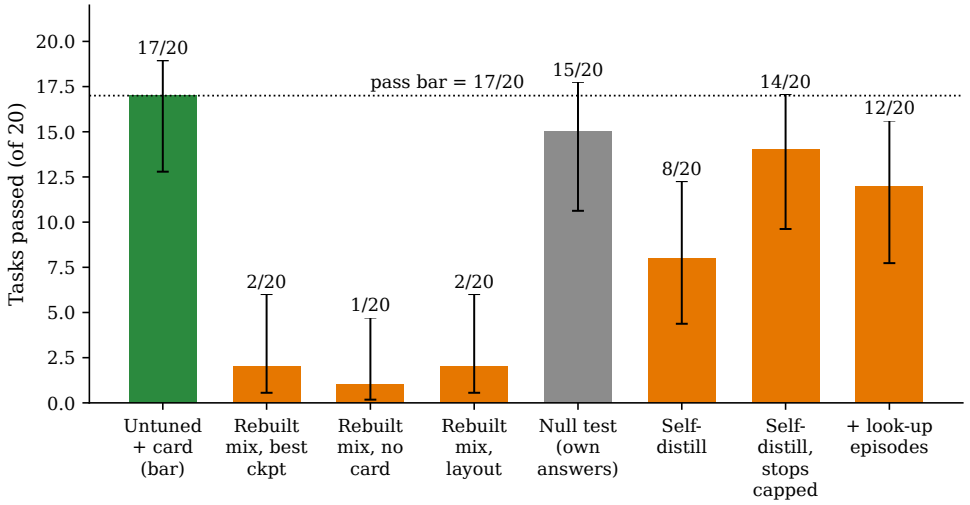


Figure 4: 20-task gate (same 20 tasks for every bar; not 200-task accuracies). Error bars: Wilson 95% intervals in tasks; run IDs in Table 6. The untuned model with the status card sets the bar (17/20). Every model trained on the rebuilt mix collapses (1–2/20). The null test stays close to the bar, so the pipeline is not the cause. Self-distillation and its two rebalanced retrains land in between and do not pass.

A candidate cause that targeted retrains did not confirm. *Self-distillation*—training on the model's own answers that the checker marked correct—scored 8/20; in 6 of 12 failures the model said an ID or price was missing and stopped instead of calling the tool that would supply it. Compared with the null-test data, look-up-type calls fell from 37.9% to 19.7% of calls and stop targets rose from 23.1% to 33.2%. Two targeted retrains tested this: capping the stop share scored 14/20, and restoring the look-up share (39.6%) scored 12/20. Neither passed, so the data are *consistent with*, but do not establish, a shift in the decision mix (stop versus continue, look up versus act) as the cause.

7 Discussion and Limitations

State, not syntax. Across conditions, the hard part is deciding the next step given the current state of the world—including whether to keep going—not writing a valid call. Harness levers that support that decision help modestly: a summary of real results, a gated stop, and a card that mostly controls whether the model acts or abstains. Levers that *constrain* the surface form hurt. Training appeared to shift the balance between acting and stopping rather than improve tracking.

The guided-template tie. A seven-step reasoning template on the native API also scored 61/200, the same as JSON text (19 tasks swap each way; it raised state mismatches to 79). So the +3.5 pp from switching the output channel may owe as much to prompt structure as to the channel, and we do not claim the channel switch alone is a gain ($p = 0.32$ in any case). The claim we do make is narrower: on top of JSON text, harness control adds +8.0 pp, and the template result does not test that ladder; whether a guided template adds to, or substitutes for, the harness levers is untested. We did not stack the harness levers on the native API.

What we recommend. For an on-device agent: JSON-in-text calls, a harness-written state summary, a gated stop rule and a status card. The full stack is the best configuration we measured (38.5%) and is greedy; if context is tight, the minimal recipe (35.5%) is a smaller fallback. Both need no training data or GPU time. Before fine-tuning, gate on task-level checks rather than validation loss. Even the best configuration completes fewer than two tasks in five; small models are not yet reliable multi-step agents on this benchmark.

Limitations.

- **One model, one split, one device, tuned in place.** All multi-turn results use Gemma-4-E2B-it on the 200-task base split on one laptop. Settings (stop threshold, card fields, mix filters) were tuned and reported on the same tasks. Other models and the harder BFCL splits are untested.
- **Headline selection.** The full stack was assembled after seeing earlier results on the same 200 tasks (tuning leakage), and its only nominally significant gain does not survive multiple-comparison correction. Its code is not yet merged (draft PR #90).
- **Context-delivered control.** The state summary and status card are injected into the model's context; "harness optimization" here includes these harness-written messages, not only parsing and stopping logic.
- **Sample size and decoding.** Intervals are about ±7 pp; greedy decoding with one quantization and one seed; the vote of five uses one sampling seed.

- **Gate design.** The 20 gate tasks were picked post hoc using historical per-task results from the same split, and the bar was set by the configuration it screens against, so the gate is a screening heuristic.
- **Fairness of comparisons.** The native baseline used eight server slots versus four elsewhere; the card and the vote were never run on plain JSON text. Per-task results come from a local evaluation database that is not in the repository.
- **Home-made constrained decoding.** Ours is a two-pass approximation on stock llama-server, so "forcing valid syntax hurts" may partly mean "our approximation hurts".
- **Training confounds.** Mix variants changed several factors at once, and self-distillation also took about 3× as many updates as the null test.

8 Future Work and Conclusion

Future work. First, *confirmatory runs*: re-score the full stack and the JSON baseline on held-out tasks, the harder BFCL multi-turn splits and a second small model, with matched serving. Second, *retrieval-augmented tool selection*: retrieve the few relevant tools (and their documentation or past usage) for each step instead of placing every schema in context [4, 22]; shorter, more relevant context could help a small model choose look-ups before actions. Third, *decision-balanced training data*: vary the look-up and stop shares while holding everything else fixed, to test the mix-shift hypothesis directly.

Conclusion. A small model running locally can pick a single tool well but fails most multi-step tasks: Gemma-4-E2B-it scores 83.1% on single calls and 27.0% on multi-turn tasks through the same native interface. Those failures are about state—tools that never ran and worlds that end wrong—not malformed output, and forcing valid output makes them worse. Optimizing the harness reached 38.5% with no sampling and no training, +8.0 points over JSON-text calls and on par with a five-sample vote at a seventh of the time; the gain is suggestive ($p = 0.023$ uncorrected) and was found on the tasks used for tuning. Fine-tuning did not reach that level (15.0% SFT, 26.5% DPO), and a null test places the fault in the data rather than the pipeline. For practitioners, the best configuration we measured is also one of the cheapest: four greedy harness changes.

References

[1] Google. Gemma 4 E2B instruct. `https://huggingface.co/google/gemma-4-e2b-it`, 2026. E2B = effective 2B.

[2] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. `https://gorilla.cs.berkeley.edu/leaderboard.html`, 2024.

[3] ggml-org. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2024.

[4] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

[5] Yalda Taheri, Mohammad Hassan Heydari, Erfan Naaman, and Afsaneh Fatemi. Small reasoning models are instruction followers in function calling. *arXiv preprint arXiv:2608.22472*, 2026.

[6] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[7] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, et al. XGrammar-2: Dynamic and efficient structured generation engine for agentic LLMs. *arXiv preprint arXiv:2601.04426*, 2026.

[8] M. Yu, K. Mu, and G. Wu. PTC-Decoder: Towards intelligent SLMs on offline resource-constrained edge devices. *arXiv preprint arXiv:2609.30836*, 2026.

[9] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. *arXiv preprint arXiv:2408.02442*, 2024.

[10] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. Acon: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.

[11] R. Guo, Y. Huang, Y. Wu, M. Zhong, Y. Chen, M. Tang, T. Long, and V. T. Hu. From state to action: OODA-Tool for reliable multi-turn tool use. *arXiv preprint arXiv:2608.24368*, 2026.

[12] Yuxuan He et al. Improving large language models function calling and interpretability via guided-structured templates. *arXiv preprint arXiv:2509.18076*, 2025.

[13] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[14] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, et al. xLAM: A family of large action models to empower AI agent systems. *arXiv preprint arXiv:2409.03215*, 2024.

[15] Qiqiang Ye et al. Hammer: Robust function-calling for on-device language models via function masking. In *International Conference on Learning Representations (ICLR)*, 2025.

[16] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Liu, et al. ToolACE: Winning the points of LLM function calling. *arXiv preprint arXiv:2409.00920*, 2024.

[17] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Zhong, et al. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *arXiv preprint arXiv:2406.18518*, 2024.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[19] Apple Machine Learning Research. MLX: An array framework for Apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[20] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[21] Sunghee Jung et al. DiaTool-DPO: Multi-turn direct preference optimization for tool-augmented large language models. *arXiv preprint arXiv:2504.02882*, 2025.

[22] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

A Intervention details

JSON in text (IFFC). Following Appendix B of [5], the tool list is omitted from the API request; the system prompt asks for a JSON list of `{function_name, parameters}` objects in assistant text. Tool results return as user messages. Calls map to the same BFCL function strings as native calling.

State summary (scratchpad). After each successful tool execution, the harness writes a compact `[STATE]` block from real results (current directory, known files, last executed calls). The last two turns stay verbatim; older raw outputs are dropped. It is not a model-written summary. Example:

```
[STATE]
cwd: /workspace/notes
files: todo.txt, log.csv
last: cd(folder='notes')->ok
```

Stop on repeats. After a tool result: (1) if the proposed call set equals the previous one, stop; (2) else if every proposed call was already executed in this user request *and* at least six non-empty tool turns have occurred, stop; (3) otherwise execute. The six-step window resets on a new user question. The ungated variant applies rule (2) with no minimum.

Status (OODA) card. Prefixed to a working copy of the latest user message (the stored transcript is unchanged), stacked on stop-on-repeats. Fields: `goal` (latest user question, truncated); `slots` (up to eight key–value facts from executed arguments and results); `last` (last executed call and outcome); `next` (advisory: `refuse` if no tools are available, `ask` if the last result signals a missing or invalid argument, `final` if the user's message is a closing "thanks/done", otherwise `call`). The hint does not constrain decoding and the card never reads reference trajectories. It is not the multi-pass OODA-Tool of [11]. Example:

```
[OODA]
goal:  lock the doors and start the engine
slots:  lockStatus=locked
last:  startEngine()->error:  brake pedal must be pressed
next:  call
```

Vote of five. On the scratchpad configuration, each step samples five completions at temperature 0.7, majority-votes the `(function_name, arguments)` tuple, executes the winner once, and continues. It is not a learned verifier.

Constrained decoding. Two requests per call: the first constrained to an empty list or a list of valid tool names, the second constraining arguments with each tool's JSON schema. No native tool grammar is active. This approximates XGrammar-2's tag-triggered dispatch [7] on stock llama-server.

Tool-name automaton. A live finite automaton restricts the tool name to valid names while arguments are free, on JSON text, not stacked on stop-on-repeats. Execution accuracy here is not comparable to the F1 reported by Yu et al. [8].

Training. LoRA adapters [18] on Gemma-4-E2B-it with mlx-lm; best-validation checkpoint fused and quantized to Q4_K_M; evaluated with JSON text. The rebuilt mix: 30,000 training / 1,500 validation rows; 45/40/10/5% simulated / APIGen / ToolACE / xLAM; 22% empty-list targets (all after a tool result); 41.2% of rows with multi-call targets; no evaluation tool names copied into simulations. DPO: β=0.2, LoRA, scored on JSON text.

B Full scoreboard, failure labels and training history

B.1 Training history

Version history. Dropping only mid-dialogue empty targets from the first mix still scored 1/200; deleting trailing empty answers at serving time scored 4/200. After the repaired mix (30/200), two further filtering variants scored 26/200 and 23/200 (Table 3); each changed several filters at once, so they are not single-factor ablations. The rebuilt mix's training was stopped early at step 26,500 of 32,000; the best validation loss (0.030) was at step 22,000, and that checkpoint is the one evaluated. Continued training for 2,500 steps on test-style tool layouts scored 2/20.

Table 3: All scored 200-task conditions in this paper (Gemma-4-E2B-it Q4_K_M). Interval: Wilson 95%. Internal IDs are in Table 6.

Condition	Correct	Acc.	95% interval
Full stack: state summary + stop + card (headline; draft PR #90)	77	38.5%	32.0–45.4
Vote of five on state summary	73	36.5%	30.1–43.4
Status card on stop-on-repeats (minimal recipe)	71	35.5%	29.2–42.3
State summary	68	34.0%	27.8–40.8
Stop on repeats (gated, six steps)	65	32.5%	26.4–39.3
JSON text (IFFC)	61	30.5%	24.5–37.2
Guided template prompt on native API	61	30.5%	24.5–37.2
Native API (NFC)	54	27.0%	21.3–33.5
Stop on first repeat (ungated)	54	27.0%	21.3–33.5
DPO, full train	53	26.5%	20.9–33.0
Best SFT (repaired mix)	30	15.0%	10.7–20.6
Tool-name automaton	27	13.5%	9.4–18.9
SFT, filters on first-turn targets off[†]	26	13.0%	9.0–18.4
Best SFT + state summary	26	13.0%	9.0–18.4
SFT, extra first-turn and state filters[†]	23	11.5%	7.8–16.7
Constrained decoding (two-pass)	21	10.5%	7.0–15.5
SFT first mix, trailing empty answers stripped at serving	4	2.0%	0.8–5.0
SFT first mixes (two variants)	1	0.5%	0.1–2.8

[†]First-turn filters drop or require training examples where the first response is empty or under-specified; state filters drop continuations inconsistent with executed tool results.

Table 4: Primary error per failed task (200 tasks). Full = full stack; Card = minimal recipe; CD = constrained decoding; Autom. = tool-name automaton; SFT = best fine-tune.

Label	NFC	IFFC	Stop	Ungated	Scratch	Card	Full	Vote	CD	Autom.	SFT
Empty turn	59	41	38	40	31	17	20	36	14	17	2
Under-call	19	38	37	31	37	41	35	38	70	54	59
State mismatch	68	60	60	75	64	71	68	53	95	102	109
Failed	146	139	135	146	132	129	123	127	179	173	170

Self-distillation counts. Look-up-type calls: 67/177 in the null-test data versus 78/395 in the self-distillation data; stop targets: 36/156 versus 146/440. Capping the stop share (22.9%) left the look-up share low (20.6%). The look-up-restoring retrain lost 7 tasks the untuned model solves, and empty turns returned (9 of 66 turns). Look-up counts rest on a name-matching heuristic.

Stop targets in the rebuilt mix. Stop targets were 22% of all rebuilt-mix rows, all after a tool result. That is a different denominator from the first mix's 78.1% (the share of targets *after a tool result* that were stops), so the two are not directly comparable. The rebuilt-mix models still under-called, so stop share alone does not explain the collapse.

10-task tripwire. Before the 20-task gate, trained models faced a disjoint 10-task check. Two fine-tuned checkpoints and the untuned model with the card all scored 3/10, passing the same three tasks, and across 11 strong historical 200-task runs its rank correlation (Spearman) with the 200-task score was 0.118 (approximate 95% interval −0.52 to 0.67). We therefore used it only as a regression tripwire, never to rank runs.

C Single-turn results (Gemma-4-E2B-it)

Table 5: Single-call BFCL v4 accuracy by category, native API. The two parallel-call categories are excluded from the headline single-call number because earlier runs on this server hit a tool-grammar bug that truncated multi-call output.

Category (n)	Correct	Accuracy
Simple (400)	375	93.75%
Multiple (200)	183	91.50%
Irrelevance (240)	214	89.17%
Java (100)	37	37.00%
JavaScript (50)	14	28.00%
Weighted, excl. both parallel categories (990)	823	83.13%

D Run-ID map, reproducibility and glossary

Table 6: Paper names and internal evaluation-log IDs (a private local database). 10- and 20-task rows are early checks, not 200-task accuracies.

ID	Name in this paper (internal name)	Result
4	Native API (NFC)	54/200
5	JSON text (IFFC)	61/200
8	Constrained decoding (CD)	21/200
9	Guided template on native API (GST+NFC)	61/200
10	State summary (IFFC+STATE)	68/200
11	Vote of five (BoN N=5 on IFFC+STATE)	73/200
12, 13	SFT first mixes (SFT-v1, SFT-v2)	1/200 each
14	Trailing empty answers stripped (SFT-v2 strip)	4/200
15	Best SFT (SFT-v4)	30/200
16	Best SFT + state summary (SFT-v4+STATE)	26/200
17, 18	SFT filter variants (SFT-v5, SFT-v6)	23/200, 26/200
27	Stop on first repeat (ungated)	54/200
30	Stop on repeats (gated, MIN=6)	65/200
42	Minimal recipe (OODA card on stop-on-repeats; “R42”)	71/200
46	Tool-name automaton (PTC-A)	27/200
51	DPO full train	53/200
69	20-task gate bar: untuned + card	17/20
70	Rebuilt mix, best-validation checkpoint (SFT-v11, step 22,000)	2/20
71	Rebuilt mix, no card	1/20
72	Rebuilt mix, layout continue-train (SFT-v11 fmt2500)	2/20
73	Null test (v13 null)	15/20
74	Self-distillation, correct turns only (v13-sd)	8/20
75	Self-distillation, stop share capped (v13-sdb)	14/20
76	+ simulated look-up episodes (v13-lk)	12/20
78	Full stack (`iffc-state-ooda`; draft PR #90 @ `c894ec6`; headline)	77/200
79	Single-call BFCL, native API	823/990

Reproducibility. Hardware: MacBook Pro, Apple M4 Pro, 48 GB. llama.cpp build 8111 (commit `11c325c6e`), `llama-server -jinja`, flash attention, full GPU offload, `-c 65536`, `-np 4` (native

baseline `-np 8`). BFCL v4 via the `bfcl-eval` package (exact version per run not logged). Temperature 0 except voting (0.7, N=5, one seed). The full stack (run 78) is `benchmark.db` `scores.id` 51 with log `sft/v13/paper_evals/runs/b_fullstack_mtb200.log`, harness code at draft PR #90 commit `c894ec6`; single-call (run 79) is `scores.id` 52–58 with log `sft/v13/paper_evals/runs/a_single_turn_gemma.log`. Failure labels are derived from the checker messages in the `eval_errors` table. Figures are regenerated from these counts by `paper/figures/src/make_refocus_figs.py`.

Table 7: Glossary.

Term	Meaning
AST	Abstract syntax tree; the single-turn BFCL matcher
BFCL	Berkeley Function Calling Leaderboard
BoN	Best-of-N; here a majority vote over five sampled calls
DPO	Direct Preference Optimization
Harness	The program around the model that parses calls, runs tools, records state and decides when a turn ends
IFFC	Instruction-followed function calling: calls written as JSON in ordinary text
LoRA	Low-rank adapters for parameter-efficient fine-tuning
NFC	Native function calling: structured `tool_calls` via the server's tool API
OODA	Observe–orient–decide–act; here, the four-line status card
pp	Percentage points
SFT	Supervised fine-tuning
SLM	Small language model
20-task gate	Fixed 20-task early check (bar 17/20); not a 200-task accuracy
10-task tripwire	Fixed 10-task regression check; does not rank models
Null test	A training run that should change nothing if the pipeline is sound
Self-distillation	Training the model on its own answers that the checker marked correct
Stop target	A training response that is an empty list: no further call
Q4_K_M	A 4-bit weight-quantization format of llama.cpp
Wilson interval	A confidence interval for a proportion that stays accurate for small counts
McNemar test	A paired test using only tasks where two conditions disagree